GUÍA PRÁCTICA DE DESARROLLO WEB

HTML + CSS + JavaScript + PHP + SQL

Un ejemplo completo, sencillo y ampliable: gestor de incidencias

Cómo utilizar esta guía

La idea no es memorizar cinco lenguajes por separado, sino entender cómo colaboran. Usaremos una pequeña aplicación para registrar incidencias: el usuario rellena un formulario, JavaScript mejora la interacción, PHP recibe y valida la petición, SQL guarda o consulta los datos, y HTML/CSS presentan el resultado.

El recorrido de los datos

1. 1. HTML: define el formulario y la tabla donde se verán las incidencias.
2. 2. CSS: organiza la pantalla y la adapta a móvil.
3. 3. JavaScript: valida rápidamente el formulario y envía la petición sin recargar toda la página.
4. 4. PHP: valida otra vez en el servidor y ejecuta la operación solicitada.
5. 5. SQL: guarda, consulta o actualiza los datos en la base de datos.
6. 6. PHP devuelve JSON; JavaScript lo recibe y actualiza el HTML.

Esquema: Navegador (HTML/CSS/JS) ⇄ petición HTTP/JSON ⇄ PHP ⇄ SQL/Base de datos.

Nota: para mantener el ejemplo didáctico y compacto se utiliza PHP con PDO y MySQL. Los nombres de tablas y rutas son ilustrativos; ajusta la conexión a tu entorno local.

1. Estructura del proyecto

```
gestor-incidencias/
├── index.html
├── styles.css
├── app.js
├── api/
│   └── incidents.php
└── database.sql
```

Cada archivo tiene una responsabilidad. Separarlos facilita mantener el código y localizar errores.

2. Base de datos — database.sql

Primero se crea la base de datos y la tabla. Cada incidencia tendrá un identificador, título, descripción, estado y fecha.

```
CREATE DATABASE IF NOT EXISTS gestor_incidencias
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;

USE gestor_incidencias;

CREATE TABLE IF NOT EXISTS incidents (
    id INT AUTO_INCREMENT PRIMARY KEY,
    title VARCHAR(120) NOT NULL,
    description TEXT,
    status ENUM('abierta', 'en_proceso', 'resuelta')
        NOT NULL DEFAULT 'abierta',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Datos de ejemplo para probar la lista.
```

```
INSERT INTO incidents (title, description, status)
VALUES
('Error de acceso', 'Un usuario no puede entrar', 'abierta'),
('Revisar formulario', 'Falta validar un campo', 'en_proceso');
```

Consultas SQL habituales

```
-- Leer todas las incidencias, de la más reciente a la más antigua.
SELECT id, title, description, status, created_at
FROM incidents
ORDER BY created_at DESC;

-- Buscar una incidencia concreta.
SELECT * FROM incidents WHERE id = 1;

-- Crear una incidencia (en PHP usaremos una consulta preparada).
INSERT INTO incidents (title, description)
VALUES ('Título de ejemplo', 'Descripción de ejemplo');

-- Cambiar el estado.
UPDATE incidents SET status = 'resuelta' WHERE id = 1;

-- Eliminar una incidencia.
DELETE FROM incidents WHERE id = 1;
```

SELECT consulta; INSERT crea; UPDATE modifica; DELETE elimina. En UPDATE y DELETE revisa siempre el WHERE. En una aplicación real, no concatenes datos del usuario dentro de una consulta SQL: usa parámetros.

3. HTML — index.html

HTML describe el contenido. Los atributos id permiten que JavaScript encuentre los elementos; el atributo action es una alternativa tradicional si se envía el formulario sin JavaScript.

```
<!doctype html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Gestor de incidencias</title>
  <link rel="stylesheet" href="styles.css">
  <script src="app.js" defer></script>
</head>
<body>
  <main class="container">
    <header>
      <h1>Gestor de incidencias</h1>
      <p>Registra y consulta incidencias del equipo.</p>
    </header>

    <form id="incident-form">
      <label for="title">Título *</label>
      <input id="title" name="title" maxlength="120" required>

      <label for="description">Descripción</label>
      <textarea id="description" name="description" rows="3"></textarea>

      <button type="submit">Crear incidencia</button>
      <p id="message" role="status" aria-live="polite"></p>
    </form>

    <section aria-labelledby="list-title">
      <h2 id="list-title">Incidencias</h2>
```

```
      <div id="incident-list" class="incident-list"></div>
    </section>
  </main>
</body>
</html>
```

label for debe coincidir con el id del campo. required aporta validación básica en el navegador, pero no sustituye la validación PHP. defer permite cargar el archivo JavaScript sin bloquear el análisis del HTML.

4. CSS — styles.css

```
/* Estilos generales */
* {
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: Arial, sans-serif;
  color: #202938;
  background: #f3f6fa;
}

.container {
  width: min(100% - 32px, 850px);
  margin: 32px auto;
}

header, form, .incident {
  background: white;
  padding: 18px;
  border-radius: 10px;
  margin-bottom: 16px;
  border: 1px solid #dce3eb;
}

label {
  display: block;
  margin: 12px 0 6px;
  font-weight: bold;
}

input, textarea, button {
  font: inherit;
}

input, textarea {
  width: 100%;
  padding: 10px;
  border: 1px solid #aab5c3;
  border-radius: 6px;
}

button {
  margin-top: 12px;
  padding: 10px 14px;
  border: 0;
  border-radius: 6px;
  cursor: pointer;
}
```

```css
.incident {
  display: flex;
  justify-content: space-between;
  align-items: start;
  gap: 12px;
}

.incident h3 {
  margin-top: 0;
}

.status {
  font-size: 0.9rem;
  color: #526174;
}

@media (max-width: 600px) {
  .incident {
    flex-direction: column;
  }
}
```

La clase .container centra el contenido; Flexbox distribuye cada incidencia; @media cambia la disposición en pantallas estrechas. Las clases se reutilizan en varios elementos.

5. PHP — api/incidents.php

Este archivo actúa como una API pequeña. Recibe el método HTTP, valida los datos y usa PDO para comunicarse con MySQL. Devuelve JSON para que JavaScript pueda leer el resultado.

```php
<?php
header('Content-Type: application/json; charset=utf-8');

// Cambia estos valores según tu entorno local.
$host = 'localhost';
$db   = 'gestor_incidencias';
$user = 'root';
$pass = '';

try {
    $pdo = new PDO(
        "mysql:host=$host;dbname=$db;charset=utf8mb4",
        $user,
        $pass,
        [PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION]
    );
} catch (PDOException $e) {
    http_response_code(500);
    echo json_encode(['error' => 'No se pudo conectar con la base de datos']);
    exit;
}

$method = $_SERVER['REQUEST_METHOD'];

try {
    if ($method === 'GET') {
        // Leer todas las incidencias.
        $stmt = $pdo->query(
            'SELECT id, title, description, status, created_at
             FROM incidents ORDER BY created_at DESC'
        );
        echo json_encode($stmt->fetchAll(PDO::FETCH_ASSOC));
```

```
        exit;
    }

    if ($method === 'POST') {
        // Leer JSON enviado desde JavaScript.
        $data = json_decode(file_get_contents('php://input'), true);

        $title = trim($data['title'] ?? '');
        $description = trim($data['description'] ?? '');

        // La validación del servidor es imprescindible.
        if ($title === '' || mb_strlen($title) > 120) {
            http_response_code(400);
            echo json_encode(['error' => 'El título es obligatorio y debe tener como máximo 120 caracteres']);
            exit;
        }

        // Los parámetros evitan concatenar la entrada del usuario en SQL.
        $stmt = $pdo->prepare(
            'INSERT INTO incidents (title, description)
             VALUES (:title, :description)'
        );
        $stmt->execute([
            'title' => $title,
            'description' => $description
        ]);

        http_response_code(201);
        echo json_encode([
            'id' => (int) $pdo->lastInsertId(),
            'title' => $title,
            'description' => $description,
            'status' => 'abierta'
        ]);
        exit;
    }

    // Este ejemplo implementa GET y POST; otros métodos aún no están implementados.
    http_response_code(405);
    echo json_encode(['error' => 'Método no permitido']);

} catch (PDOException $e) {
    // En producción, registra el detalle en el servidor, no lo muestres al usuario.
    http_response_code(500);
    echo json_encode(['error' => 'Error al procesar la operación']);
}
```

PDO prepara la consulta y envía los valores por separado. Nunca devuelvas al usuario contraseñas, consultas internas ni detalles sensibles de excepciones. Para producción también se necesitan autenticación, autorización, configuración segura de credenciales y controles adicionales.

6. JavaScript — app.js

JavaScript escucha el formulario, llama a PHP mediante fetch y actualiza la pantalla sin recargarla.

```
const form = document.querySelector('#incident-form');
const titleInput = document.querySelector('#title');
const descriptionInput = document.querySelector('#description');
const list = document.querySelector('#incident-list');
const message = document.querySelector('#message');
```

```
const API_URL = 'api/incidents.php';

// Cargar los datos guardados al abrir la página.
async function loadIncidents() {
  try {
    const response = await fetch(API_URL);

    if (!response.ok) {
      throw new Error('No se pudieron cargar las incidencias');
    }

    const incidents = await response.json();
    renderIncidents(incidents);
  } catch (error) {
    message.textContent = error.message;
  }
}

// Construir la lista usando textContent para mostrar texto de forma segura.
function renderIncidents(incidents) {
  list.replaceChildren();

  incidents.forEach(incident => {
    const article = document.createElement('article');
    article.className = 'incident';

    const content = document.createElement('div');
    const title = document.createElement('h3');
    const description = document.createElement('p');
    const status = document.createElement('p');

    title.textContent = incident.title;
    description.textContent = incident.description || 'Sin descripción';
    status.className = 'status';
    status.textContent = `Estado: ${incident.status}`;

    content.append(title, description, status);
    article.append(content);
    list.append(article);
  });
}

// Crear una incidencia al enviar el formulario.
form.addEventListener('submit', async event => {
  event.preventDefault();

  const title = titleInput.value.trim();
  const description = descriptionInput.value.trim();

  if (!title) {
    message.textContent = 'Escribe un título.';
    return;
  }

  try {
    const response = await fetch(API_URL, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ title, description })
    });

    const result = await response.json();
```

```
    if (!response.ok) {
      throw new Error(result.error || 'No se pudo crear la incidencia');
    }

    message.textContent = 'Incidencia creada correctamente.';
    form.reset();
    await loadIncidents();
  } catch (error) {
    message.textContent = error.message;
  }
});

// Ejecutar al cargar la página.
loadIncidents();
```

JSON.stringify convierte un objeto JavaScript a texto JSON; response.json() convierte la respuesta JSON en datos JavaScript. fetch no considera automáticamente los estados 400 o 500 como excepciones, por eso comprobamos response.ok.

7. Cómo se conectan las cinco tecnologías

7. HTML crea #incident-form y #incident-list.
8. CSS aplica los estilos a .container, .incident y los campos.
9. JavaScript encuentra el formulario con querySelector y escucha submit.
10. JavaScript envía title y description a api/incidents.php mediante POST con JSON.
11. PHP lee el cuerpo de la petición, valida los datos y ejecuta un INSERT parametrizado.
12. MySQL guarda el registro y PHP devuelve un JSON con estado HTTP 201.
13. JavaScript comprueba response.ok, vuelve a consultar los datos y actualiza la lista.

8. Mejoras para practicar después

- Añadir un filtro por estado usando un SELECT con WHERE.
- Crear un desplegable para elegir prioridad y añadir una columna priority a SQL.
- Implementar PATCH o una ruta específica para marcar una incidencia como resuelta.
- Implementar DELETE con confirmación y una consulta preparada.
- Añadir un buscador que filtre por título.
- Mostrar un mensaje cuando no haya incidencias.
- Deshabilitar el botón mientras se envía el formulario.
- Separar la configuración de la base de datos del archivo público de la API.
- Añadir autenticación y comprobar permisos antes de modificar datos.

9. Fallos típicos y cómo localizarlos

Síntoma	Qué comprobar
La página no tiene estilos	Ruta de styles.css y pestaña Network del navegador.
JavaScript no encuentra el formulario	Que el id coincida y que el script se cargue correctamente.
La petición devuelve 404	La URL API_URL y la ubicación del archivo PHP.
La petición devuelve 500	Registros de PHP, conexión y errores SQL.
La respuesta no es JSON válido	Que PHP no imprima avisos antes del JSON y que la respuesta sea realmente JSON.
No se insertan datos	Método POST, cuerpo JSON, validación y consulta preparada.

Los datos desaparecen al recargar	Que se estén guardando en la base de datos y no solo en un array JavaScript.

10. Conceptos rápidos de entrevista

¿Qué es CRUD? Create, Read, Update, Delete: crear, consultar, actualizar y eliminar.

¿Por qué validar en PHP si ya valido en JavaScript? El cliente se puede manipular; el servidor debe aplicar las reglas de seguridad.

¿Qué es una clave primaria? Un campo o conjunto de campos que identifica de forma única una fila.

¿Qué es una clave foránea? Una referencia a una clave de otra tabla, que permite relacionar registros.

¿Qué es una API? Una interfaz que permite que dos sistemas se comuniquen mediante reglas definidas.

¿Qué diferencia hay entre GET y POST? GET suele consultar; POST suele enviar datos para crear o procesar un recurso.

¿Qué hace async/await? Permite escribir código asíncrono basado en promesas con una sintaxis más fácil de seguir.

¿Qué evita una consulta preparada? Reduce el riesgo de inyección SQL al separar la consulta de los valores.

¿Qué es responsive? Una interfaz que se adapta a diferentes tamaños de pantalla.

¿Qué es Git? Un sistema de control de versiones para registrar y compartir cambios.

11. Git: flujo normal de trabajo

```
git status
git diff
git add index.html styles.css app.js api/incidents.php database.sql
git commit -m "Crea gestor básico de incidencias"
git pull
git push
```

Antes de hacer commit, revisa git diff y git status. En un equipo, integra los cambios remotos según el flujo de trabajo acordado. No subas contraseñas, tokens ni archivos de configuración con credenciales.

12. Mini examen para hacer sin mirar

14. En JavaScript, filtra un array de incidencias para obtener solo las que tienen status === 'abierta'.
15. Escribe una consulta SQL que cuente las incidencias por estado.
16. Explica qué ocurre desde que se pulsa Crear hasta que el registro aparece en la pantalla.
17. ¿Por qué no se debe confiar solo en required del HTML?
18. ¿Qué diferencia hay entre textContent e innerHTML?
19. ¿Qué harías si el servidor devuelve 500?
20. Añade a la tabla una columna priority con un valor por defecto.
21. ¿Qué método HTTP usarías para consultar, crear, modificar y eliminar?

Soluciones orientativas:

```
// 1.
const openIncidents = incidents.filter(i => i.status === 'abierta');

// 2.
SELECT status, COUNT(*) AS total
FROM incidents
GROUP BY status;

// 7. Ejemplo SQL para MySQL:
```

```
ALTER TABLE incidents
ADD COLUMN priority INT NOT NULL DEFAULT 1;
```

Para las preguntas conceptuales, responde describiendo el flujo, la validación y la responsabilidad de cada capa. Intenta hacer el mini examen primero sin consultar la guía.

13. Cómo ejecutar el ejemplo

22. Necesitas un entorno con PHP, MySQL y un servidor web (por ejemplo, XAMPP si ya lo utilizas).
23. Crea la base de datos ejecutando database.sql en tu gestor de MySQL.
24. Coloca los archivos dentro de la carpeta pública del servidor; mantén api/incidents.php dentro de api/.
25. Actualiza las credenciales de conexión de PHP para tu equipo.
26. Abre index.html a través del servidor web, no directamente como archivo file://.
27. Comprueba en las herramientas de desarrollador del navegador la pestaña Network y la consola si algo falla.

Limitaciones didácticas: esta versión implementa listar y crear. Las operaciones de actualizar y eliminar aparecen como SQL y ejercicios de ampliación, pero aún no están expuestas como endpoints PHP. Tampoco incluye autenticación, autorización, protección CSRF ni configuración de producción.